Snakes and Ladders

Final Report

Yasmine Mohamed: 900242127

Talia Mebed: 900232496

Michael Henin: 900212000

Sandra Fraige: 900242170

Yassin Ragab: 900241197

Applied data structures

The American University in Cairo

Semester: Fall 2025

Abstract:

This project implements a complete Snakes and Ladders game using Breadth First Search (BFS) on a graph represented through an adjacency list to compute the minimum number of dice rolls required to reach the final square (100). The project integrates a custom graph engine, queue structure, random board generator, and a fully interactive Qt-based GUI supporting multiplayer gameplay, smooth animations, board editing, and shortest-path visualization.

Keywords:

Snakes and Ladders, BFS, adjacency list, queue, shortest path, graph traversal, Qt GUI, board engine, pathfinding

1.1 Introduction

Snakes and Ladders is a well-known board game traditionally played using dice and a 100-square board containing snakes that move players backward and ladders that move players forward. Although simple in appearance, the game can be mapped directly to core concepts in computer science such as graph modeling, pathfinding, and algorithmic optimization.

In this project, we redesign Snakes and Ladders as a computational problem rather than simply a recreational game. By representing the board as a graph, where each square corresponds to a vertex and each dice roll corresponds to an edge, the game becomes a shortest-path problem. The objective is to determine the minimum number of dice throws needed to reach the final square (100) starting from the initial square (1).

Our implementation is built using Qt C++, which enables a fully interactive graphical interface, multi-player functionality, and smooth game animation. Beyond gameplay, the project integrates the Breadth First Search (BFS) algorithm to compute the shortest path through the board. This blend of interactive design and algorithmic computation allows the project to serve both as an engaging game and as an educational demonstration of graph traversal techniques.

1.2 Problem Definition

The core problem in this project is determining the minimum number of dice rolls needed to reach square 100 in a Snakes and Ladders game. To solve this analytically, the board is modeled as a directed graph where:

- Each cell is a vertex.
- Each dice outcome (1–6) forms a directed edge to a reachable cell.
- Snakes and ladders override normal movement by redirecting edges to new vertices.

These nonlinear transitions create a search space with many possible routes. Instead of relying on randomness, the goal is to compute the exact shortest path for any board configuration. This requires correctly modeling redirected edges, handling boundary conditions, and updating the graph whenever the board changes.
BFS is used to guarantee the optimal solution in terms of the fewest dice rolls.

1.3 Methodology

The project models the Snakes and Ladders board as a directed graph and uses several structured steps to compute the shortest path and build the full application:

1. **Graph Modeling**
    - Each of the 100 board cells is treated as a vertex.
    - Dice rolls (1–6) define edges to reachable cells.
    - Snakes and ladders redirect edges to their endpoints.
    - The graph is stored using an adjacency list.
2. **Data Structures Implementation**
    - A custom Queue is implemented for BFS operations.
    - A BFS solver is developed to compute shortest paths.
    - Both structures are integrated into BoardEngine.

3. **Game Engine (BoardEngine)**
   - Builds and rebuilds the graph based on the current board.
   - Applies snakes and ladders during movement.
   - Calls BFS and reconstructs the optimal path and dice rolls.
4. **Qt Graphical User Interface**
   - BoardWidget handles board drawing and animations.
   - GameWindow manages gameplay, dice rolls, path display, statistics, and logic flow.
   - Includes multiplayer support, board editor, and logging panel.
5. **Random Board Generation**

   The RandomBoard module generates snake and ladder placements and saves them to CSV.

   Ensures valid, non-overlapping board configurations for testing.

6. **Testing & Evaluation**
   - Multiple scenarios were tested: random boards, custom boards, and multiplayer games.
   - Verified shortest path accuracy, animation correctness, and state transitions.

   Methodology:
   1. We first represented the board as a directed graph with 100 cells.
   2. Then, during the first milestone we constructed a queue (array based) and adjacency matrix (2D dynamic array based) classes in order to use them in implementing their BFS. More details can be found in the pseudocode section.
   3. Then, after the first milestone we started implementing the UI on Qt where we included multiple features

1.4 Specification of Algorithms to be Used

The BFS algorithm is used to traverse the graph and determine the optimal sequence of moves. The adjacency list represents all valid transitions. Dice reconstruction is performed after obtaining the final path.

Pseudocode (GeeksforGeeks, 2025) :

Adjacency list:

```
Class adj_list:
private: v=100, adj[][], board[]

adj_list(snakes[], ladders[]):
board[i]=i for i=0..99
board[snake_start-1]=snake_end-1 for each snake
board[ladder_start-1]=ladder_end-1 for each ladder
for i=0..99:
deg = i≤93?6:99-i
adj[i][j] = board[i+j+1] for j=0..deg-1

~adj_list(): delete adj[], board
getNeighbors(v): return adj[v]
getDegree(v): v≤93?6:99-v
```

BFS using the adjacency list and queue:

```
Initialize queue Q
Enqueue start node (cell 1) into Q
Mark start node as visited
while Q is not empty:
current = Dequeue from Q
for dice_roll in 1 to 6:
next_cell = current + dice_roll
if next_cell contains ladder or snake:
next_cell = endpoint of ladder or snake
if next_cell == destination:
return shortest path length
if next_cell not visited:
Enqueue next_cell into Q
Mark next_cell as visited
```

1.5 Data Specifications

1. **Board of fixed size:** 10×10 = 100 cells
2. **Input Format:**
- Snakes and ladders defined as pairs of integers (start position, end position) using 2D arrays.
- Randomly generated boards stored in CSV as where we find the snakes and ladders positions.
3. **Graph Structure:**
- Adjacency matrix of size 100×100
- Lists all valid edges based on dice outcomes
4. **Player Data:**
- Current position, number of turns, transitions
5. **Algorithm Output:**
- Shortest path sequence
- Number of dice rolls
- Board display with snakes and ladders positions

1.6 Experimental Results

Tests were conducted using random boards, custom boards, and various player counts. BFS consistently computed shortest paths. GUI functions such as animations, updates, and statistics were validated successfully.

Experiments were conducted on multiple board configurations.

Test Case	Expected output
Test 1: Empty board without snakes and ladders	Path with 17 moves (dice rolls of 6)
Test 2: One huge ladder from 2 →100	2 moves: 1 → 2 → 100
Test 3: Ladder chain 2 →20 and 20 →100	3 moves 1→ 2 →20 → 100
Test 4: Snake from 14 → 3 and ladder from 4→ 50	Snake should be avoided and ladder at 4 should be used
Test 5: Snake: 95 → 10, Ladder: 3 → 90	Snake should be avoided and use the ladder, the minimal number of moves should be 3

Test 6 (for UI): Multiplayer Progression	Statistics correctly tracked per player. GUI highlighted whose turn it was.

All tests were passed.

1.7 Analysis and Critique

Strengths

- **Modular design**: Board engine independent from UI.
- **Efficient BFS solver**: Computes optimal path in milliseconds due to fixed graph size.
- **Robust GUI**: Supports animation, editing, logs, dynamic stats.
- **Real-world software principles**: event-driven UI, object-oriented design, reusable components.
- **Random board generator** ensures replayability.

Weaknesses / Limitations

- Board size is fixed; scaling to NxN would require minor refactoring.
- BFS only provides an optimal path in terms of dice rolls, not probabilistic strategies.
- Graph rebuilt after every board change (not expensive but could be optimized).
- No AI opponent included.

Future Enhancements

- Add AI players with expected-value strategy.

- Include sound effects and theme packs for GUI.
- Implement networked multiplayer.
- Allow users to save/load custom boards.

1.8 Conclusions

This project demonstrates the practical application of data structures, graph theory, and algorithm design in building a fully interactive game. By modeling Snakes and Ladders as a directed graph and applying BFS, we efficiently determine the shortest path under any board configuration. The integration of Qt GUI components with backend logic showcases strong software engineering practices. Overall, the project achieves its objectives and provides a versatile platform for exploring algorithmic behavior in a real-time environment.

Acknowledgements

We thank Dr.Ashraf Abdelraouf, our Applied Data Structures instructor, and teaching assistants for guidance throughout the project. We also acknowledge open-source Qt documentation and BFS educational resources that supported our development.

Appendix:

Code listings for BFS, Queue, BoardEngine, BoardWidget, GameWindow, WelcomePage, RandomBoard, StatsFormatter, main.cpp.

References:

1. GeeksforGeeks. (2025, October 25). *Breadth first search or BFS for a graph*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/